Lab 6: Code and Branch Coverage Analysis

Warranty Claims (warranty_claims.py)

Name	SRN
RISHIT V N S	PES1UG24AM225

1. Objective

In this lab I measured how much of warranty_claims.py is tested using the coverage tool with pytest. Starting from the two given test cases, I wrote more tests to get line coverage and branch coverage above 90%, and looked at why 100% line coverage does not always mean every path has been tested.

2. Environment and Commands

Item	Details
Python	3.11.0 (Windows, virtual environment)
Tools	pytest 9.2.0, coverage 7.17.0
Clear old data	`python -m coverage erase`
Line coverage	`python -m coverage run -m pytest <test_file>.py`
Branch coverage	`python -m coverage run --branch -m pytest <test_file>.py`
Report	`python -m coverage report -m --include=warranty_claims.py`

--include=warranty_claims.py makes the report show only the module being tested, not the test file itself. -W ignore was added when running to hide a warning about coverage's C tracer, which was blocked by the system policy on my laptop. Coverage then uses its Python tracer instead, so the results are the same.

3. Module Under Test

Function	What it does	Decisions (if/else)
`out_of_warranty_charge`	Works out the repair charge from the plan, product age, claim type and registration.	Unknown plan, negative age, registration extension, in warranty, accidental vs defect, excess over 12 months, extra accidental surcharge
`claim_allowed`	Checks whether a new claim can be filed.	Recall or not, less than 30 days since last claim, number of claims
`sla_status`	Says how close a claim is to its service deadline.	Hardware or not, priority or not, on track / at risk / critical

The module has 37 statements and 26 branches.

4. Starter Tests (Before)

The given file test_warranty_claims.py had only 2 tests.

```
python -m coverage run --branch -m pytest test_warranty_claims.py
2 passed

Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
warranty_claims.py       37     23     26      6    32%   6, 8, 11, 15-23, 28, 30, 35-45
```

Line coverage was 38% and branch coverage was 32%. 23 of the 37 lines were never run: both error checks, the registration extension, all of the charge calculation, the recall and 30-day rules, and the whole sla_status function.

5. Code Coverage Analysis

Test file: CodeCoverageTest_warranty_claims.py (12 tests). The aim here was to make every line run at least once, so I added one test for each part of the code that was missing.

Test	Lines it covers
Unknown plan / negative age	The two ValueError lines (6, 8)
Registered accidental claim	Registration extension (11) and accidental rate (16)
Defect just out of warranty	Defect rate (18) and the final charge
Accidental, long out of warranty	Surcharge for excess over 12 months (20, 22)
Recall claim / claim too soon	Recall rule (28) and the 30-day rule (30)
Hardware on track / at risk, software priority critical	All of sla_status (35-45)

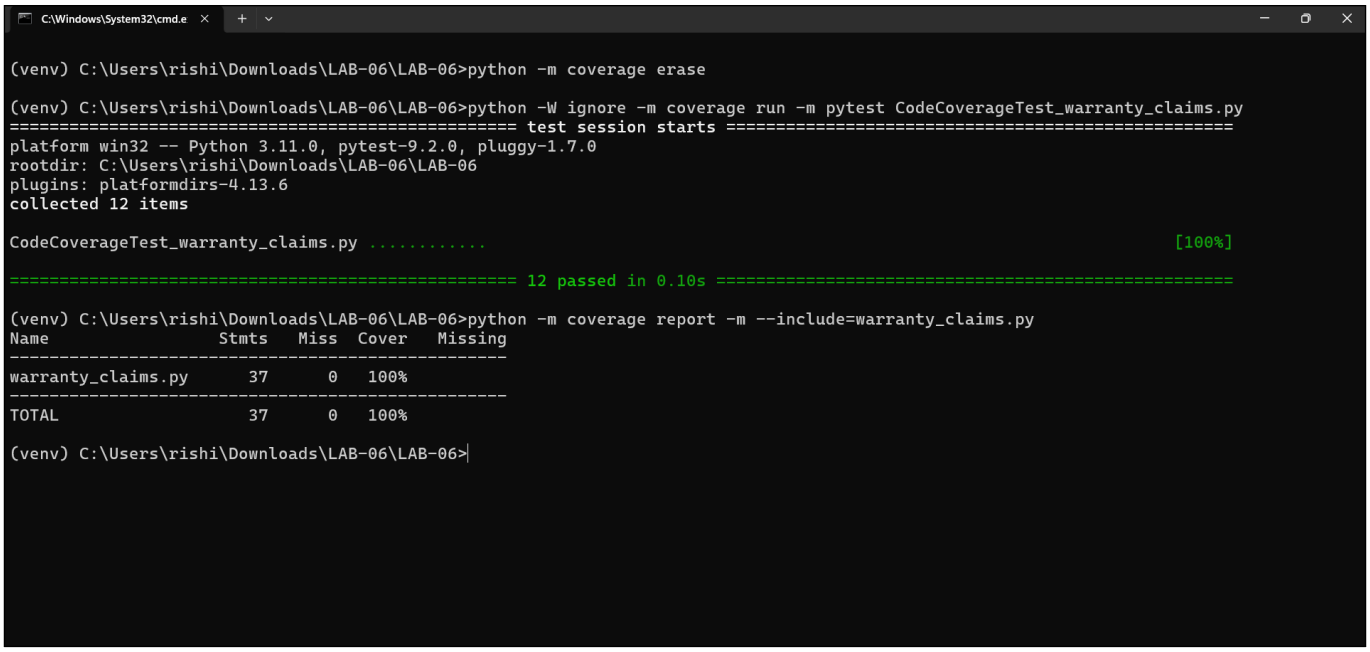


Figure 1: Code coverage result (12 passed, 37 statements, 0 missed, 100%)

Line coverage went from 38% to **100%**. But when I ran these same 12 tests with --branch, one branch was still missing:

```
python -m coverage run --branch -m pytest CodeCoverageTest_warranty_claims.py
12 passed

Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
warranty_claims.py       37      0     26      1    98%   21->23
```

21->23 means the check on line 21 (claim_type == "accidental", inside excess > 12) was only ever True. Line 22 ran, so every line was covered, but no test had a long excess with a defect claim. So line coverage said 100% even though one path through the code was never tested.

6. Branch Coverage Analysis

Test file: BranchCoverageTest_warranty_claims.py (27 tests). Here the aim was to make every if/else go both ways, and also to test the boundary values where the result changes.

Function	Branch / boundary tested	Test input	Expected
out_of_warranty_charge	Missing branch 21->23: long excess with a defect claim	basic, 25, defect	1950

Function	Branch / boundary tested	Test input	Expected
out_of_warranty_charge	Premium does not get +6 months when registered	premium, 37, defect, registered	100
out_of_warranty_charge	Age 0 is valid / excess exactly 0 is free	basic, 0 / premium, 36	0 / 0
out_of_warranty_charge	Excess exactly 12 has no surcharge, 13 does	basic, 24 / basic, 25	1200 / 1950
claim_allowed	Recall limit: 5 allowed, 6 not	(5, 5, recall) / (5, 6, recall)	True / False
claim_allowed	Recall ignores the 30-day rule	(0, 1, recall)	True
claim_allowed	29 days too soon, 30 days allowed	(29, 0) / (30, 2)	False / True
claim_allowed	More than 2 claims refused	(60, 3)	False
sla_status	Hardware: on track at 48, at risk at 36, critical at 35	48 / 36 / 35	each status
sla_status	Software: 24 / 12 / 11	24 / 12 / 11	each status
sla_status	Priority hardware cutoff is 72	72 / 71	on_track / at_risk

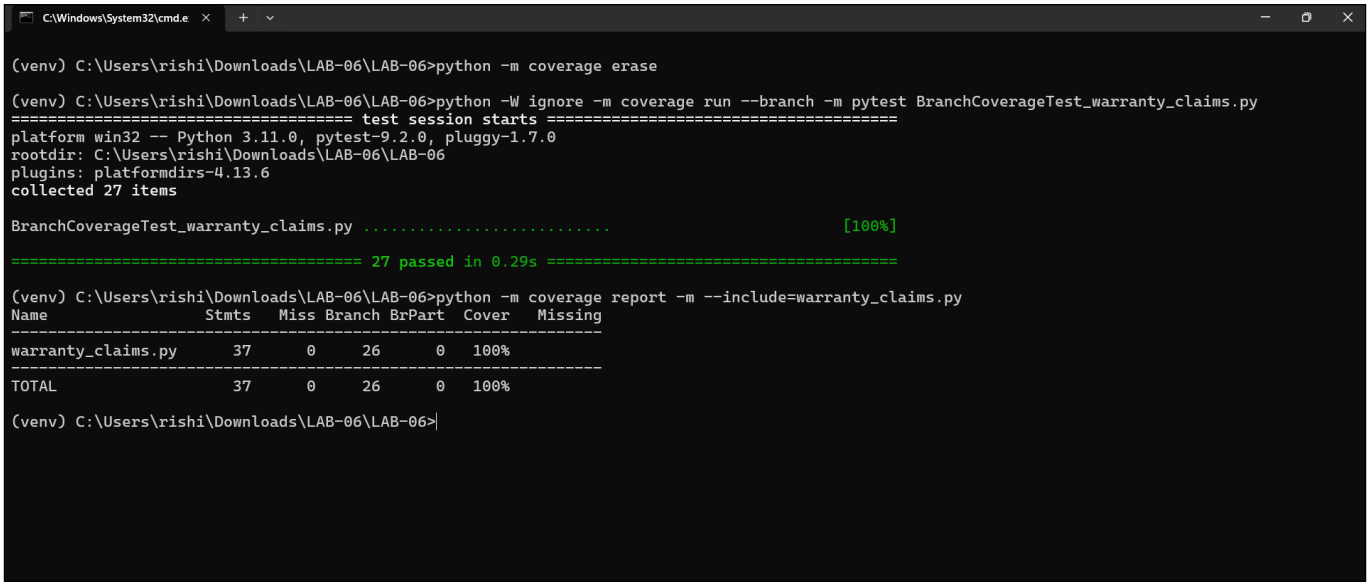


Figure 2: Branch coverage result (27 passed, 26 branches, 0 partial, 100%)

All 37 statements and all 26 branches are covered, with 0 partial branches, so branch coverage is **100%**.

7. Results Summary

Test file	Tests	Line coverage	Branch coverage
test_warranty_claims.py (starter)	2	38%	32%
CodeCoverageTest_warranty_claims.py	12	100%	98%
BranchCoverageTest_warranty_claims.py	27	100%	100%

Both targets from the lab (at least 90% code coverage and at least 90% branch coverage) were met.

8. Commits

#	Commit	Files
1	Lab 6: add code coverage tests (100% line coverage)	CodeCoverageTest_warranty_claims.py, code_coverage.png
2	Lab 6: add branch coverage tests (100% branch coverage)	BranchCoverageTest_warranty_claims.py, branch_coverage.png
3	Lab 6: add coverage analysis report	Coverage_Analysis.pdf

9. Conclusion

The biggest thing I learned is that 100% line coverage does not mean everything is tested. The code coverage tests ran every line, but one decision only ever went one way, and that only showed up with --branch. Testing at the boundaries, like exactly 30 days or exactly 12 months over, was also useful because those are the places where an off-by-one mistake in the code would change the result. To check this, I changed a few conditions in the code on purpose, for example < 30 to <= 30 and > 12 to >= 12, and each time at least one branch test failed.